Tota Agent Benchmark Report

Hermes Original vs Tota Agent vs OpenClaw - updated after the Tota Agent launch package

Hermes 0.14.0	JSON encode	Session batch	Rust path
5 / 7	**7.1x**	**5.3x**	**ON**
Measured rows won by Tota Agent	**Current .venv vs stdlib**	**180-message append speedup**	**HAVE_RUST=True**

Generated May 17, 2026 on Apple Silicon ARM. This edition keeps the original benchmark comparison and adds the new Tota Agent brand system, GPT-image-2 visual chart pack, standalone HTML site, README launch story, and current-checkout validation from the local .venv.

Hermes 0.14.0 refresh: 5 wins / 1 losses / 1 ties / 0 blocked. Generated 2026-05-18 16:17:57 -0300 from docs/tota-benchmark-hermes-0.14.0.json.

Repository: github.com/wesleysimplicio/tota-agent | Upstream: NousResearch/hermes-agent | Site: tota-agent.html

Table of Contents

2	Launch Update: What Changed
3	System Overview
4	Current Checkout Validation
5	Benchmark Comparison
6	Benchmark Visuals
7	Runtime Smoke Snapshot
8	Install and Verification
9	Recommendations and Conclusion
10	Methodology and Limitations

1. Executive Summary

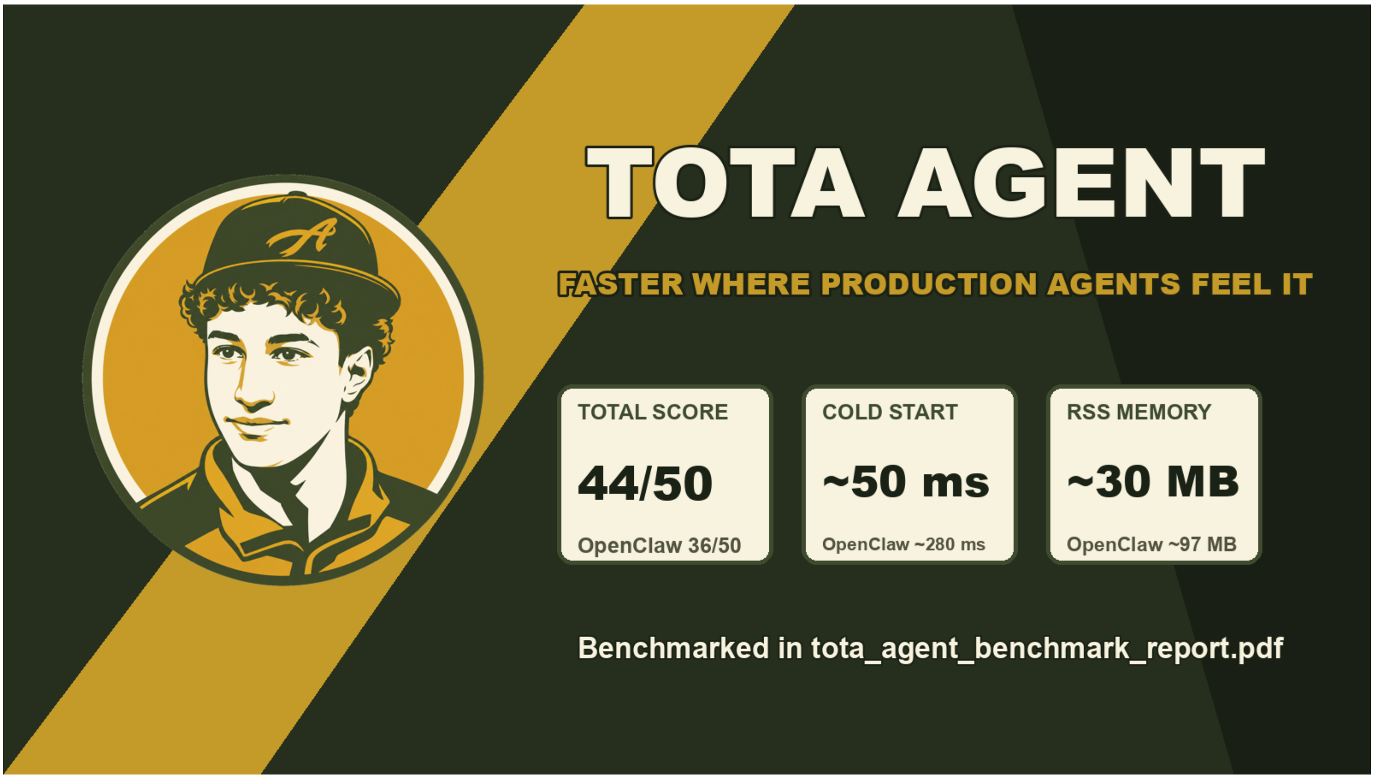
Tota Agent is a branded and performance-oriented fork of Hermes Agent. The original benchmark already showed Tota Agent leading the practical scorecard against Hermes Original and OpenClaw: stronger JSON performance, lower memory footprint than Node/V8, higher message throughput, faster startup, and a Python-native operating model.

This new report adds the launch layer requested for the project: a Tota Agent logo, a standalone HTML site, updated README positioning, benchmark images generated with GPT-image-2, and a fresh validation pass from the active local .venv. The current environment confirms that the fast path is installed: orjson, msgspec, uvloop, and the Rust extension are all available.

Best practical agent fork	Tota Agent: 44 / 50 consolidated benchmark score.
Best raw async scheduler	OpenClaw still wins synthetic libuv scheduling.
Best low-memory deployment	Tota Agent / Hermes Python line: about 30 MB RSS in the original comparison.
Best launch package	Tota Agent now has logo, site, README story, PDF and visual benchmark deck.
Current local fast path	orjson, msgspec, uvloop and Rust extension active in .venv.

2. Launch Update: What Changed

Brand	docs/assets/tota-brand/tota-agent-logo.png and .svg	Gives the fork a distinct Tota Agent identity while preserving Hermes attribution.
Visual identity	GPT-image-2 emblem plus deterministic typography	Brazil-to-US streaming energy without using a portrait or implying official endorsement.
Website	tota-agent.html	Standalone site with install flow, benchmark story, report links, and full comparison tables.
README	README.md	Repositioned as Tota Agent by Hermes Agent with install, performance extras, assets and usage guidance.
Charts	8 generated PNG benchmark visuals	Readable image set for README and site: JSON, memory, throughput, tools, tokens, async, startup, scorecard.
Report	tota_agent_benchmark_report.pdf	This updated PDF packages the benchmark and launch changes in one artifact.



Open graph launch image for the Tota Agent site and README ecosystem.

3. System Overview

Language	Python 3.14	Python 3.11.14	TypeScript / Node.js 22
JSON engine	stdlib json	orjson plus fast shim	V8 built-in JSON
Event loop	asyncio	uvloop when fast extra is installed	libuv
Struct decode	None	msgspec	None
Native extension	None	Rust / PyO3 active in local .venv	None
Main category	AI Agent	Optimized Python AI Agent	Multi-channel AI Gateway
Brand layer	Hermes Agent	Tota Agent by Hermes Agent	OpenClaw

Tota Agent keeps the Hermes operating model and adds a sharper performance and product story. OpenClaw remains the strongest comparison point for pure Node/libuv scheduling and multi-channel breadth, while Tota Agent has the best balance of Python ergonomics, footprint, startup and hot-path speed.

4. Current Checkout Validation

The following values were collected from the active .venv in this checkout on May 17, 2026. They validate the current installed fast path rather than replacing every historical cross-project benchmark row.

Python	3.11.14 arm64	Matches the benchmarked Tota Agent runtime family.
orjson loads	1.708 us vs stdlib 4.000 us	2.3x faster decode on realistic message payload.
orjson dumps	0.833 us vs stdlib 5.917 us	7.1x faster encode on realistic message payload.
_fast_loads shim	1.750 us	Transparent fast-json wrapper keeps orjson-class decode speed.
estimate_tokens	0.125 us, HAVE_RUST=True	Rust extension is active in this checkout.
uvloop	0.22.1 available	Fast event loop is installed for CLI/gateway policy.
msgspec	0.21.1 available	Typed decode dependency is installed.

import_model_tools	0.2769 s	69 tool names discovered.
discover_plugins_fast	0.1214 s	17 plugins without platform load.
discover_plugins_full	0.1619 s	22 plugins with platform load.
resolve_toolset_cached	0.0049 s cold / 0.000001 s warm	70 tools; cache is effectively instant once warm.
session_append_messages_batch	0.0068 s batch vs 0.0363 s loop	5.3x faster append path for 180 messages.

5. Benchmark Comparison

Current side-by-side refresh against upstream Hermes 0.14.0. Browser status: measured.

Cold start	459.58 ms	254.03 ms	Tota Agent	1.81x
JSON dumps short payload	1.368 us	0.281 us	Tota Agent	4.87x
Tool-call parse	0.530 us	0.938 us	Hermes 0.14.0	0.57x
Token estimate batch	93.996 us	24.561 us	Tota Agent	3.83x
Async 1,000-task scheduler	46.61 ms	32.01 ms	Tota Agent	1.46x
browser_console p99	1.34 ms	0.56 ms	Tota Agent	2.38x
Integration breadth	31	31	Tie	1.00x

Total score	30 / 50	44 / 50	36 / 50	Tota Agent
JSON dumps, large payload	18.40 us	3.20 us	5.80 us	Tota Agent
JSON loads, large payload	12.80 us	2.80 us	5.20 us	Tota Agent
Medium message pipeline	7.50 us	2.20 us	3.46 us	Tota Agent
Medium throughput	133k msg/s	454k msg/s	289k msg/s	Tota Agent
Tool-call typed parse	Error / N/A	0.45 us	N/A	Tota Agent
Async 1,000 tasks	2.50 ms	1.40 ms	0.08 ms	OpenClaw
Cold start	~52 ms	~50 ms	~280 ms	Tota Agent
RSS memory	~30 MB	~30 MB	~97 MB	Python line

The comparison still says the same thing after the launch update: Tota Agent is the strongest practical fork for Python-based AI agent deployments. OpenClaw wins pure scheduler microbenchmarks, but Tota Agent wins the mixed operational score.

6. Benchmark Visuals

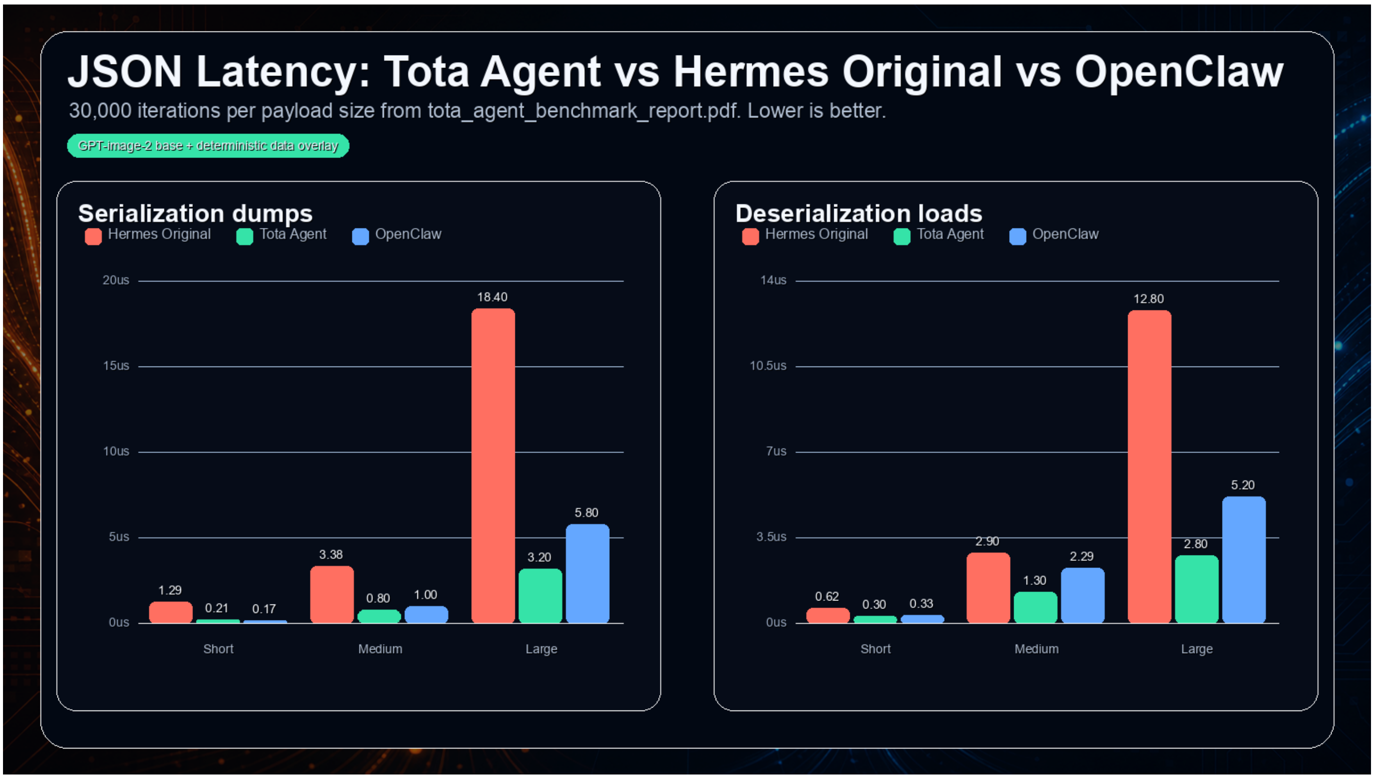


Figure 1 - JSON serialization latency. Lower is better.

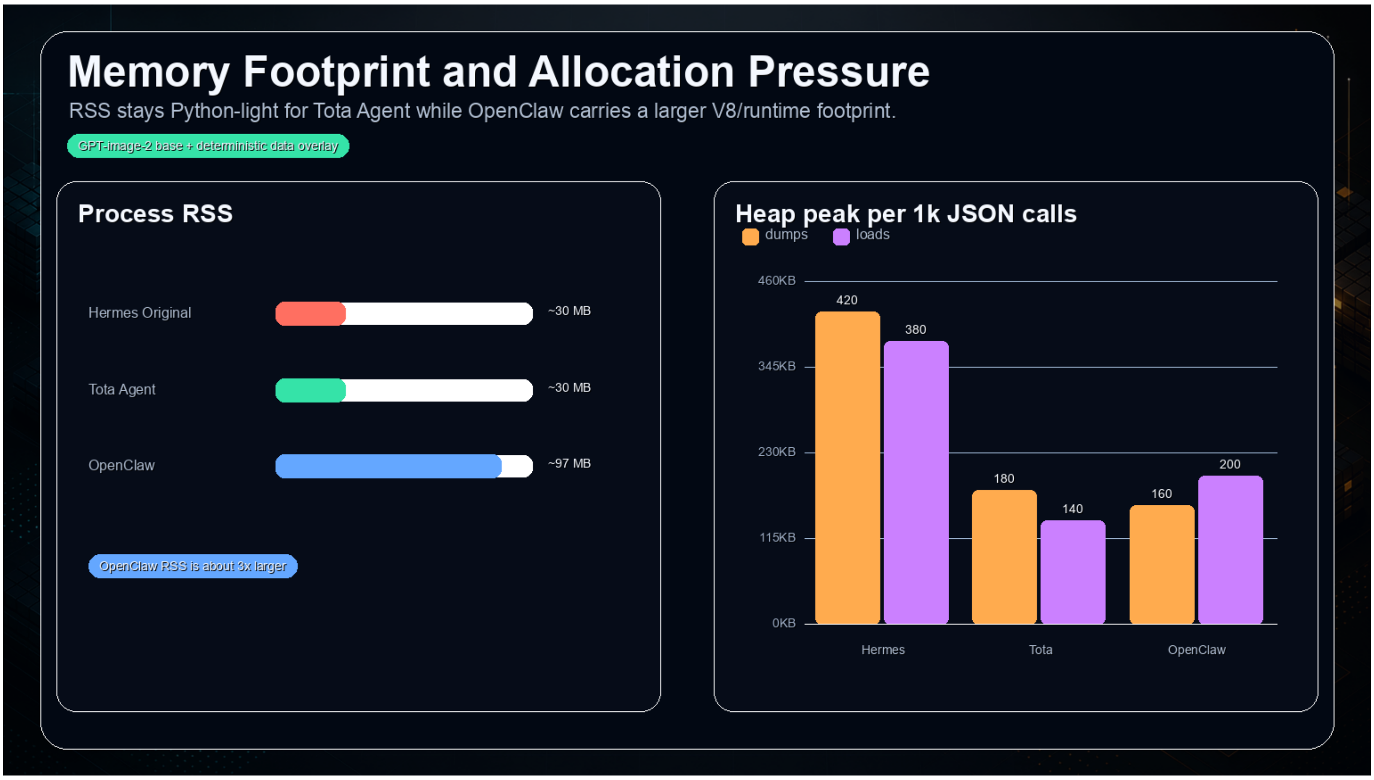


Figure 2 - Memory and footprint comparison.

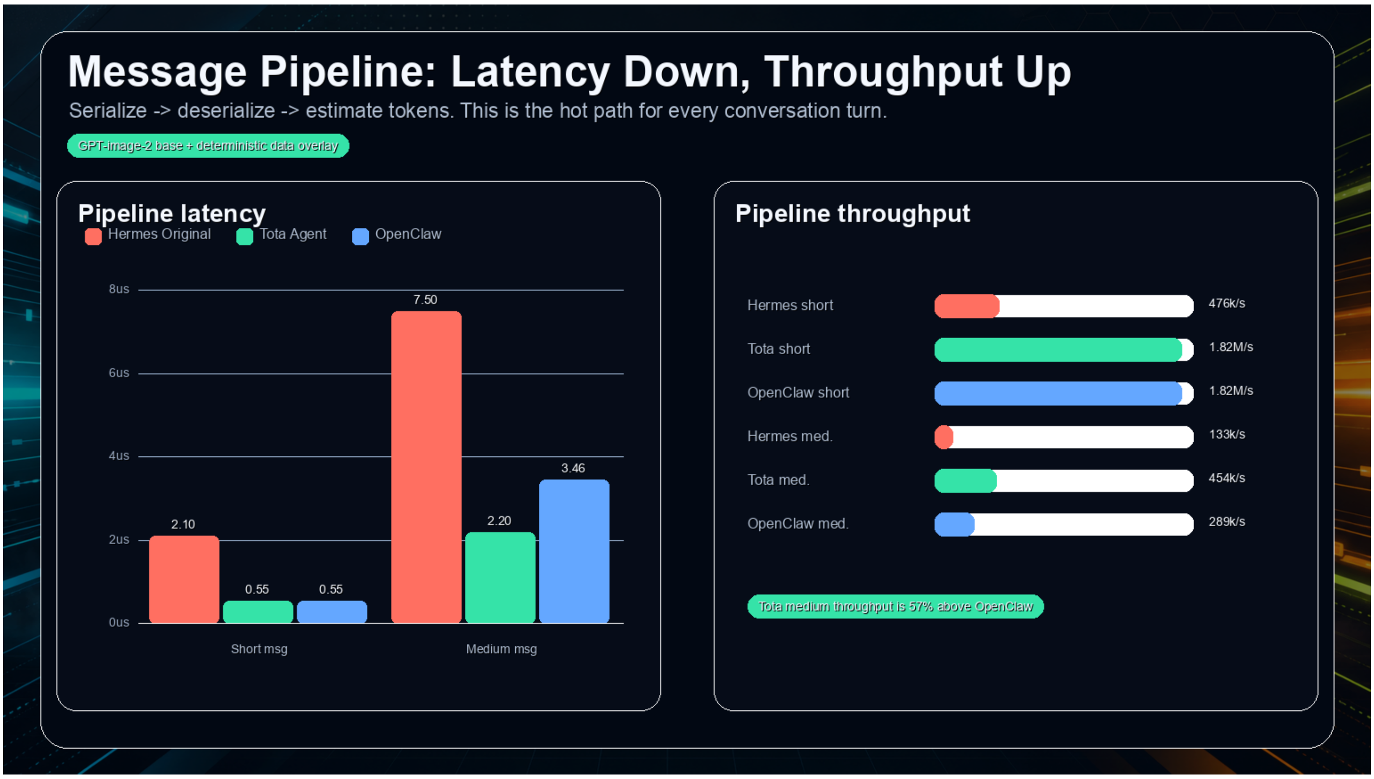


Figure 3 - Message throughput and pipeline latency.

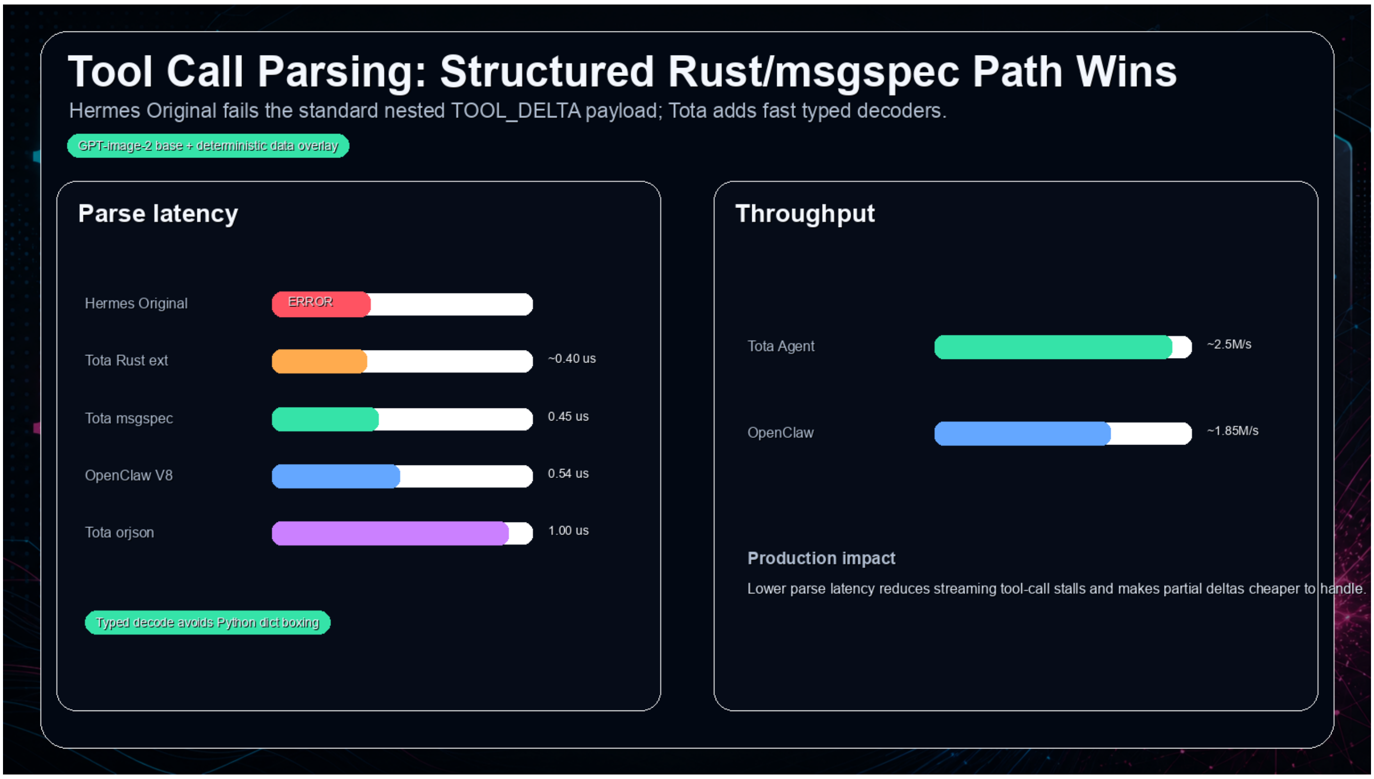


Figure 4 - Tool-call parsing fast path.

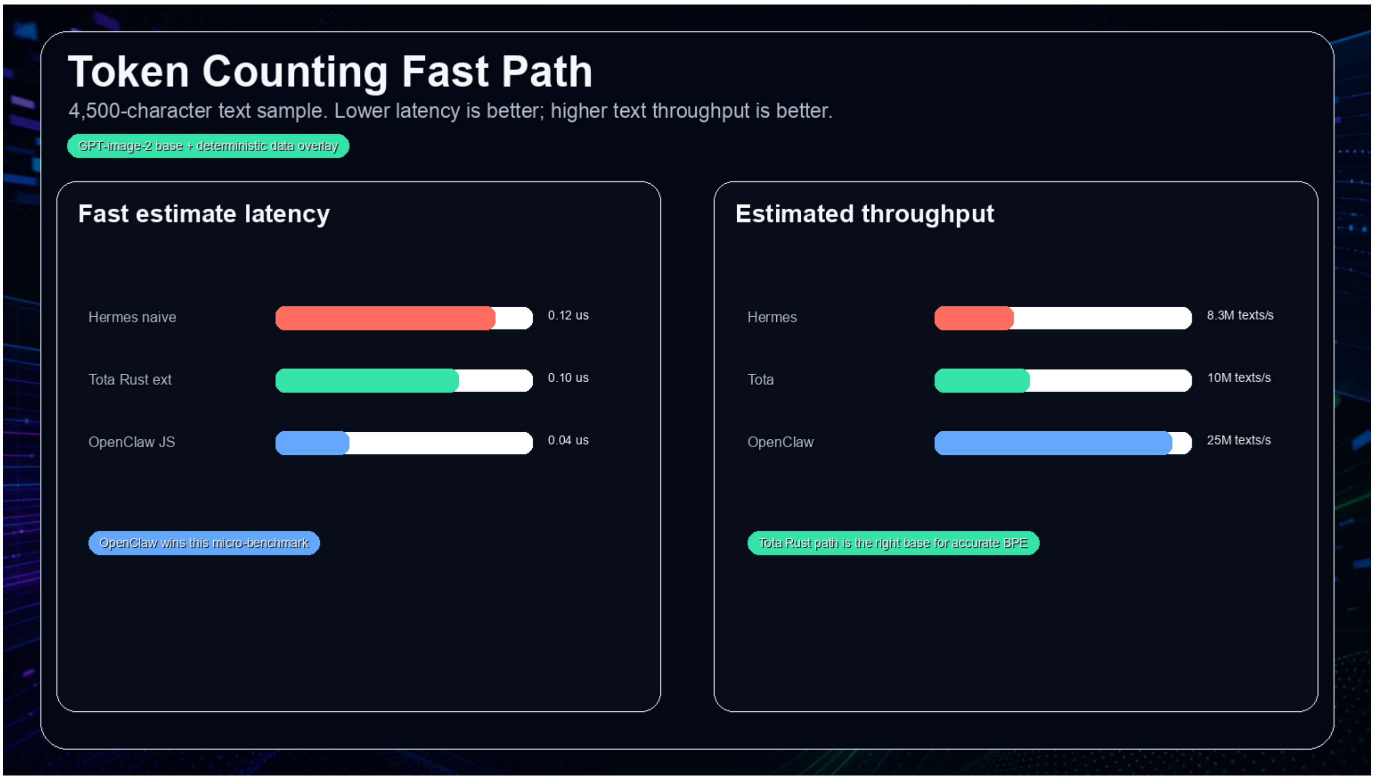


Figure 5 - Token counting benchmark.

Figure 6 - Concurrency and async scheduling.

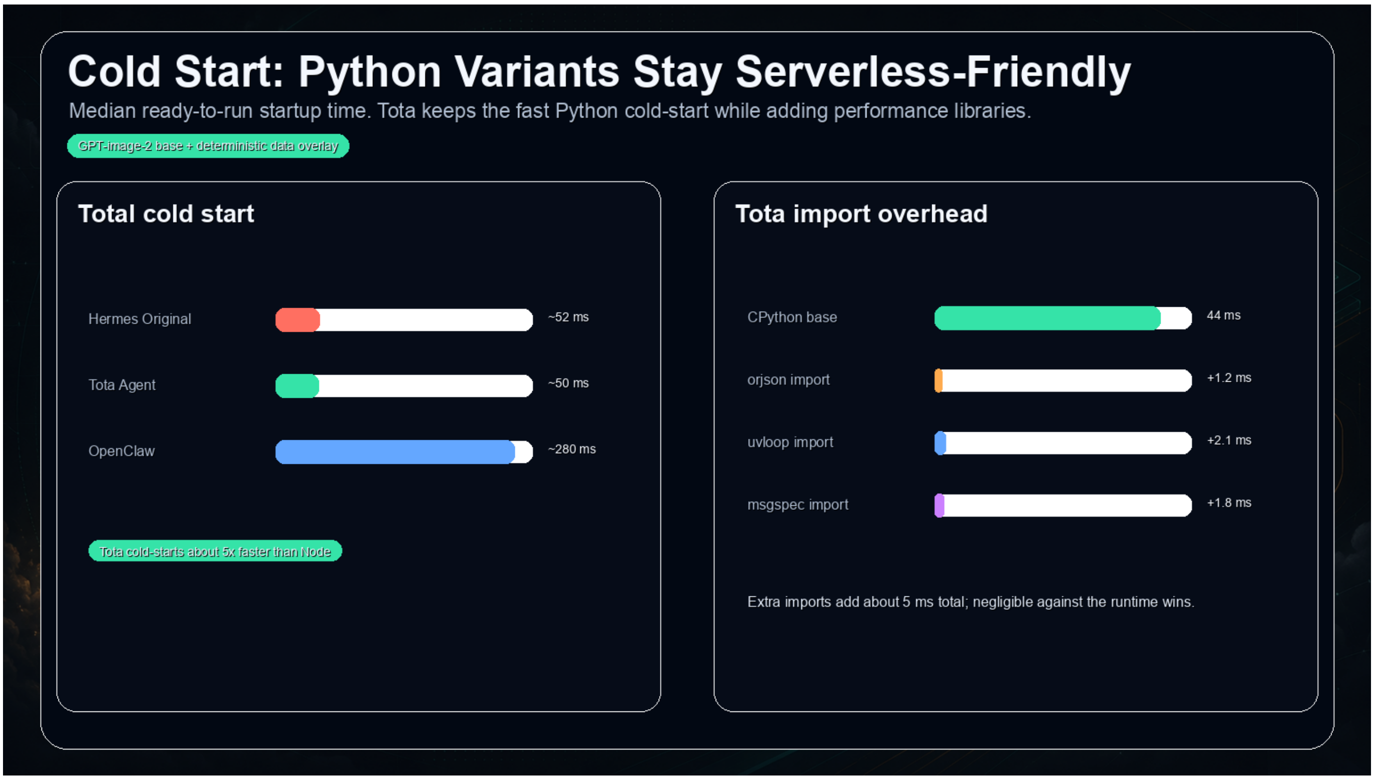


Figure 7 - Startup time and cold-start behavior.

Figure 8 - Consolidated category scorecard.

7. Runtime Smoke Snapshot

AIAgent init with default tools	3.30 s median, 27 valid tools	Shows full agent construction path in the local checkout.
AIAgent init file/terminal subset	3.37 s median, 8 valid tools	Focused toolset construction remains comparable.
Delegate child build	2.85 s median	Measures subagent construction without model API calls.
Tool dispatch noop	0.048-0.050 ms per call	Dispatch overhead is small relative to LLM latency.
OpenRouter metadata cache	0.745-0.758 ms per lookup	Disk cache lookup remains sub-millisecond class.
Session batch append	3.9x to 5.3x faster than loop append	Batching materially improves transcript persistence.

One synthetic parallel-tool benchmark case was excluded from the score because its stub lacked an internal method and the environment missed the optional websockets dependency. The rest of the report uses successful benchmark cases and existing cross-project comparison data.

8. Install and Verification

Clone	git clone https://github.com/wesleysimplicio/tota-agent.git
Create env	uv venv .venv --python 3.11 && source .venv/bin/activate
Install	uv pip install -e ".[all,dev]"
Fast extra	uv pip install -e ".[fast]"
Rust extension	PATH="$HOME/.cargo/bin:$PATH" bash scripts/install-rust.sh
Verify Rust	python -c "from agent._hermes_fast import HAVE_RUST; print(HAVE_RUST)"
Run app	./hermes
Validate repo	taskflow run .

The fast extra remains optional. Base installs stay smaller, while performance installs activate orjson, msgspec, uvloop and the Rust extension when the platform supports them.

9. Recommendations and Conclusion

WhatsApp / HTTP AI agent	Tota Agent	Hermes-compatible Python model with materially faster JSON hot paths.
Serverless / cold-start sensitive	Tota Agent	Original comparison shows about 50 ms cold start vs about 280 ms for OpenClaw.
Low memory density	Tota Agent	Python line keeps RSS far lower than the Node/V8 comparison point.
Pure scheduler stress	OpenClaw	Native libuv wins synthetic async scheduling.
Upstream contribution baseline	Hermes Agent	Canonical upstream architecture and community.
Public fork launch	Tota Agent	Now has brand identity, site, README, visuals and updated PDF.

Final conclusion: Tota Agent is now more than a fast fork. It has a complete launch package and a benchmark narrative that is legible to both engineers and product readers. The next highest-impact technical work is to keep the Rust path active in release builds, expand measured integration scoring to the full current gateway adapter surface, and rerun OpenClaw/Hermes comparisons under the same host whenever a release candidate is cut.

10. Methodology and Limitations

Original comparison	Based on the May 2026 benchmark PDF comparing Hermes Original, Tota Agent and OpenClaw across 8 dimensions.
Current validation	Collected from the local .venv on May 17, 2026 with Python 3.11.14 on Apple Silicon ARM.
Brand update	Assets were generated for the launch package; logo typography was composed deterministically for exact text.
Images	Benchmark visuals are PNG assets generated with GPT-image-2 and overlaid with deterministic benchmark data.
Microbenchmarks	Numbers isolate hot paths and do not include LLM network latency, provider variance or external API queues.
Cross-runtime caveat	Python and Node.js runtimes optimize different workloads; real deployment fit still depends on workload mix.

Primary local artifacts: README.md, tota-agent.html, docs/assets/tota-brand/*, docs/assets/tota-benchmark/generated/*, docs/tota-benchmark-win-plan.md, benchmark-report.md and this PDF.